

Compte Rendu de TP : Détection par Deep Learning - Mise en œuvre de YOLO

Introduction

Ce TP a pour objectif d'explorer la détection d'objets à l'aide du Deep Learning, en se focalisant sur l'architecture YOLO (You Only Look Once). Contrairement aux approches classiques, YOLO effectue la détection en une seule passe sur l'image, ce qui le rend particulièrement rapide et adapté au traitement en temps réel. Nous avons utilisé OpenCV (module DNN) pour l'inférence de modèles pré-entraînés, développé des interfaces Python pour manipuler les résultats, et enfin expérimenté l'entraînement d'un modèle personnalisé via Roboflow et Ultralytics.

Partie 1 : Prise en main de YOLO dans le module DNN d'OpenCV

Question 1.1 : Classes reconnues et tests des réseaux

Les modèles YOLO standards (YOLOv3, YOLOv4, etc.) sont généralement pré-entraînés sur le dataset **COCO** (Common Objects in Context). Ce dataset contient **80 classes** distinctes, allant des personnes et véhicules (voitures, vélos) aux animaux (chiens, chats) et objets du quotidien (chaises, écrans, tasses).

Différence observée entre YOLOv3 et YOLOv3-tiny :

- **YOLOv3-tiny** : Le traitement est extrêmement rapide (idéal pour un flux vidéo fluide sur une machine modeste). Cependant, la précision est moindre, particulièrement sur les petits objets ou les objets partiellement masqués.
- **YOLOv3 (complet)** : Le temps d'inférence est plus long, mais la détection est nettement plus robuste, avec des boîtes englobantes (bounding boxes) plus précises et une meilleure confiance globale.

Question 1.2 : Utilité des "thresholds" (seuils)

Lors de l'inférence, YOLO utilise principalement deux seuils critiques :

1. **Confidence Threshold (Seuil de confiance)** : Il permet d'ignorer les détections dont la probabilité est trop faible. Par exemple, si on le fixe à 0.5, le réseau ignorera tout objet dont il n'est pas sûr à au moins 50%.
2. **NMS Threshold (Non-Maximum Suppression)** : YOLO prédit souvent plusieurs boîtes englobantes pour un même objet. L'algorithme NMS conserve la boîte ayant la

plus haute confiance et supprime toutes les autres boîtes qui la chevauchent significativement (selon le seuil NMS défini par l'IoU - *Intersection over Union*).

Question 1.3 : Fichiers liés au réseau de neurones

L'utilisation de YOLO avec OpenCV DNN nécessite généralement trois fichiers :

- `yolov3.weights` : Contient les **poids** du réseau. Ce sont les valeurs (matrices) apprises par les neurones lors de la phase d'entraînement.
- `yolov3.cfg` : Fichier de **configuration**. Il décrit l'architecture exacte du réseau (nombre de couches, types de filtres, paramètres d'hyper-apprentissage).
- `coco.names` : Fichier texte contenant le **nom des classes** (une classe par ligne, l'index correspondant à l'ID de sortie du réseau).

Question 1.4 : Architecture de YOLOv3 et YOLOv3-tiny

- **YOLOv3** repose sur une architecture d'extraction de caractéristiques appelée **Darknet-53** (composée de 53 couches convolutives avec des blocs résiduels). Le réseau effectue des prédictions à 3 échelles différentes pour mieux détecter les petits, moyens et grands objets.
- **YOLOv3-tiny** est une version allégée. Il utilise un réseau beaucoup plus petit (moins de couches convolutives, pas de blocs résiduels profonds) et ne prédit qu'à 2 échelles différentes.

Dépendance au nombre de classes :

Les couches qui dépendent du nombre de classes sont les **dernières couches convolutives** (juste avant les couches d'output YOLO). Le nombre de filtres de ces couches est défini par la formule : $\text{filtres} = (\text{nombre_de_classes} + 5) * 3$ (les 5 correspondant aux coordonnées x, y, w, h et au score de confiance, multipliés par les 3 "anchors" ou boîtes de base par échelle).

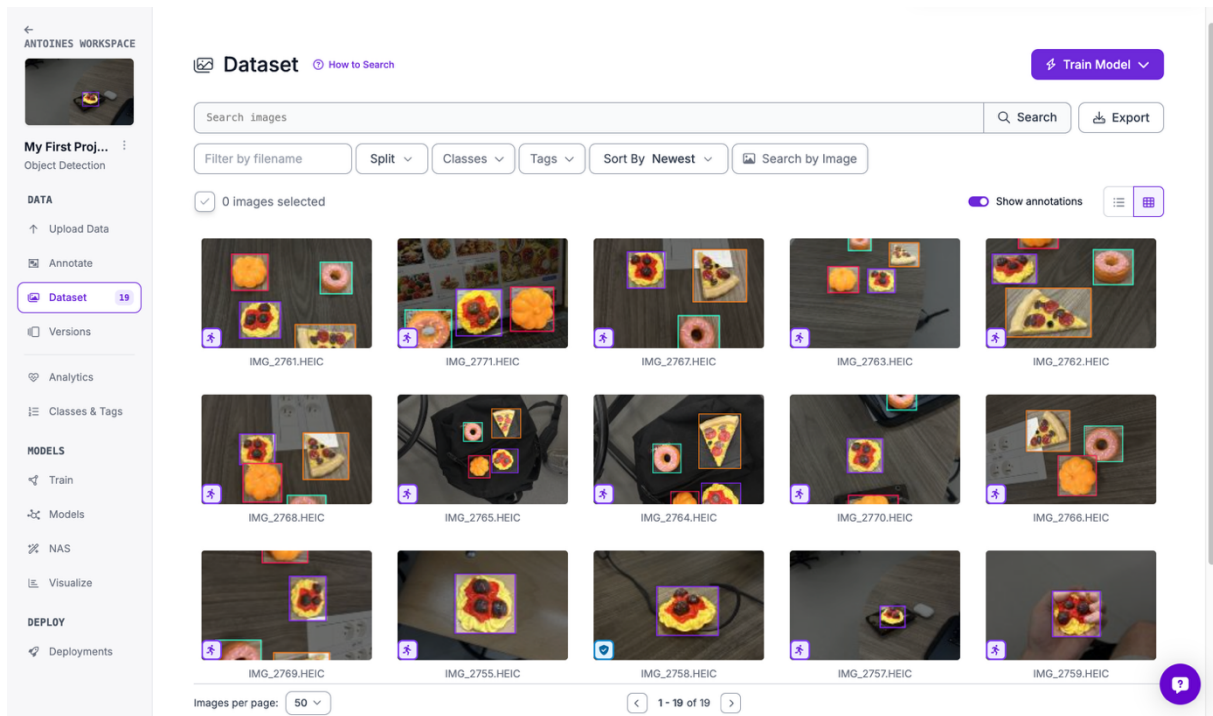
Question 1.5 : Autres modèles pré-entraînés

De nombreux autres modèles existent. Nous pouvons citer **YOLOv4, YOLOv5, YOLOv8** (disponibles au format ONNX), ou encore les modèles de la famille **MobileNet-SSD** (optimisés pour mobile). Les tests sur YOLOv8 montrent par exemple une précision accrue et un temps d'inférence très optimisé par rapport à YOLOv3.

Partie 3 : Préparation à l'entraînement de votre propre classificateur

Création du dataset

J'ai pris environ 20 photos pour 5 objets différents, en variant les fonds, les conditions d'éclairage et en introduisant des objets parasites pour éviter le surapprentissage (overfitting).



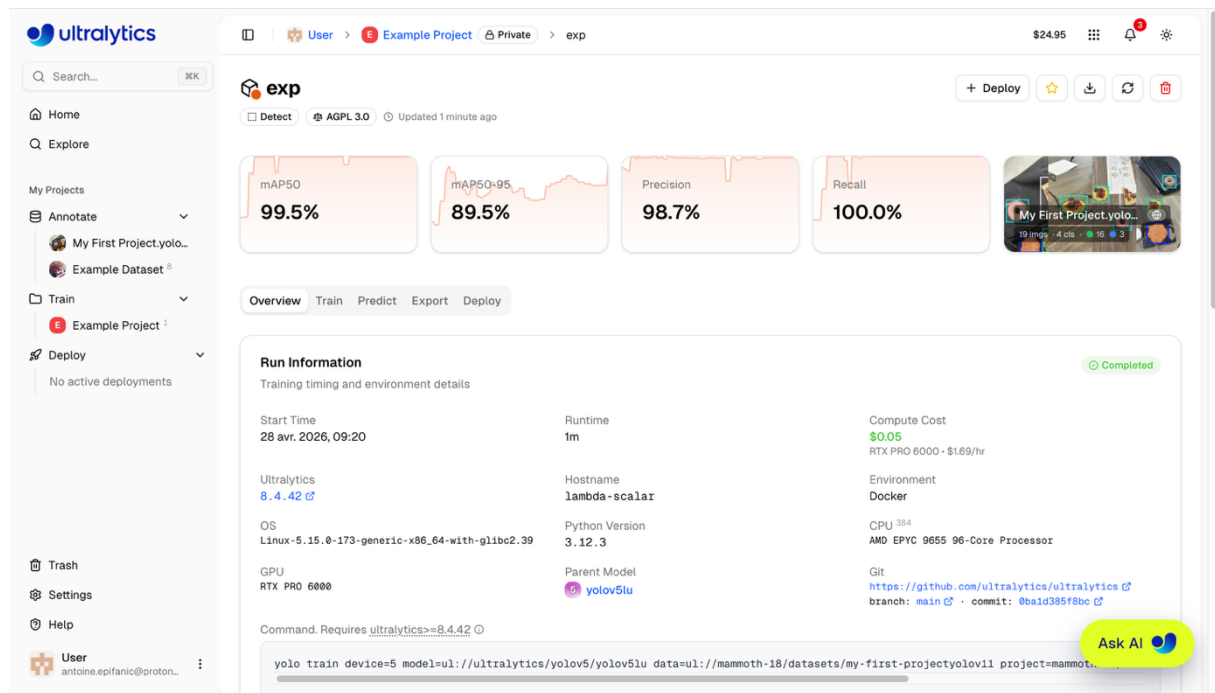
Question 3.1 : Utilisation de Roboflow

Les images ont été importées dans **Roboflow**. L'outil d'annotation permet de dessiner les boîtes englobantes très facilement. J'ai ensuite appliqué des étapes de pré-traitement (redimensionnement à 640x640) et d'augmentation de données (rotations légères, variations de luminosité) pour tripler la taille du dataset. L'entraînement a été lancé directement sur les serveurs de Roboflow (Roboflow Train).

- **Résultats** : L'inférence web montre que le modèle reconnaît bien les objets sur des images de validation.
- **Récupération** : Le réseau peut être utilisé via l'API REST de Roboflow, ou en téléchargeant les poids générés au format de notre choix pour une utilisation hors ligne.

Question 3.2 : Entraînement sous Ultralytics (Google Colab) et Export ONNX

Le dataset a été exporté depuis Roboflow au format YOLOv5 PyTorch. J'ai ensuite utilisé Google Colab pour bénéficier d'un GPU gratuit.



Question 3.3 : Utilisation du modèle personnalisé en Python

Je n'ai pas réussi à faire fonctionner la librairie ultralytics sur mon python malheureusement.